



1 Print Hello World

Print a literal string on standard output

```
std::cout << "Hello World" << std::endl;
```

```
print("Hello World")
```

Alternative implementation:

```
print('Hello World')
```

2 Print Hello 10 times

Loop to execute some code a constant number of times

```
for (int i = 0; i < 10; ++i)
    cout << "Hello\n";
```

```
for _ in range(10):
    print("Hello")
```

Alternative implementation:

```
print("Hello\n"*10)
```

Alternative implementation:

```
i = 0
while i < 10:
    print('Hello')
    i += 1
```

3 Create a procedure

Like a function which doesn't return any value, thus has only side effects (e.g. Print to standard output)

```
void finish(char* name){
    cout << "My job here is done. Goodbye " << name << "\n";
}
```

```
def finish(name):
    print(f'My job here is done. Goodbye {name}')
```

4 Create a function

Create a function which returns the square of an integer

```
int square(int x){
    return x*x;
}
```

```
def square(x):
    return x*x
```

Alternative implementation:

```
def square(x):
    return x**2
```

5 Create a 2D Point data structure

Declare a container type for two floating-point numbers *x* and *y*

```
struct Point{
    double x;
    double y;
};
```

```
@dataclass
class Point:
    x: float
    y: float
```

Alternative implementation:

```
Point = namedtuple("Point", "x y")
```

6 Iterate over list values

Do something with each item *x* of the list (or array) *items*, regardless indexes.

```
for(const auto &x : items) {
    doSomething(x);
}
```

```
for x in items:
    doSomething( x )
```

Alternative implementation:

```
[do_something(x) for x in items]
```

7 Iterate over list indexes and values

Print each index *i* with its value *x* from an array-like collection *items*

```
std::size_t i = 0;
for(const auto &x: items) {
    std::cout << "Item " << i++ << " = "
    << x << std::endl;
}
```

```
for i, x in enumerate(items):
    print(i, x)
```

Alternative implementation:

```
for (std::size_t ix = 0; ix < items.size(); ++ix) {
    std::cout << "Item " << ix << " = " <<
    items[ix] << std::endl;
}
```

8 Create a map**(associative array)**

Create a new map object **x**, and provide some (key, value) pairs as initial content.

```
std::map<const char*, int> x;
x["one"] = 1;
x["two"] = 2;
```

Alternative implementation:

```
std::unordered_map<std::string, double>
mymap = {
    {"mom", 5.4},
    {"dad", 6.1},
    {"bro", 5.9}
};
```

```
x = {"one" : 1, "two" : 2}
```

9 Create a Binary Tree data structure

The structure must be recursive because **left** child and **right** child are binary trees too. A node has access to children nodes, but not to its parent.

```
struct binary_tree
{
    int data;
    binary_tree *left = nullptr, *right =
    nullptr;
};
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

Alternative implementation:

```
class Node:
    def __init__(self, data, left_child,
    right_child):
        self.data = data
        self._left_child = left_child
        self._right_child = right_child
```

10 Shuffle a list

Generate a random permutation of the elements of list **x**

```
std::random_shuffle(x.begin(), x.end
());
```

```
shuffle(x)
```

Alternative implementation:

```
random.shuffle(x)
```

11 Pick a random element from a list

The list **x** must be non-empty.

```
std::mt19937 gen;
std::uniform_int_distribution<size_t> u
id (0, x.size () - 1);
x[uid (gen)];
```

```
random.choice(x)
```

Alternative implementation:

```
std::ranges::sample(x, &result, 1, st
d::mt19937{std::random_device{}}());
```

12 Check if list contains a value

Check if the **list** contains the value **x**. **list** is an iterable finite container.

```
bool Contains(const std::vector<int> &l
ist, int x)
{
    return std::find(list.begin(),
    list.end(), x) != list.end();
}
```

```
x in list
```

Alternative implementation:

```
auto contains(auto list, auto x) -> boo
l {
    return std::ranges::find(list, x) !=
    std::ranges::end(list);
}
```

13 Iterate over map keys and values

Access each key k with its value x from an associative array *mymap*, and print them.

```
for (const auto& kx: mymap) {
    std::cout << "Key: " << kx.first
    << " Value: " << kx.second << std::endl;
}
```

```
for k, v in mymap.items():
    print(k, v)
```

Alternative implementation:

```
for (const auto& [k, x]: mymap) {
    std::cout << "Key: " << k << "
    Value: " << x << '\n';
}
```

Alternative implementation:

```
for (auto entry : mymap) {
    auto k = entry.first;
    auto x = entry.second;
    std::cout << k << ": " << x << "\n";
}
```

14 Pick uniformly a random floating point number in [a..b)

Pick a random number greater than or equals to a , strictly inferior to b .
Precondition : $a < b$.

```
float pick(float a, float b)
{
    std::default_random_engine generator;
    std::uniform_real_distribution
    distribution(a, b);
    return distribution(generator);
}
```

```
random.uniform(a,b)
```

15 Pick uniformly a random integer in [a..b]

Pick a random integer greater than or equals to a , inferior or equals to b .
Precondition : $a < b$.

```
std::mt19937 gen;
std::uniform_int_distribution<size_t> uid
(a, b);
uid (gen);
```

```
random.randint(a,b)
```

17 Create a Tree data structure

The structure must be recursive. A node may have zero or more children. A node has access to its children nodes, but not to its parent.

```
template<typename T>
struct Node{
    T value;
    std::vector<Node<T>> children;
};
```

```
class Node:
    def __init__(self, value, *children):
        self.value = value
        self.children = list(children)
```

18 Depth-first traversal of a tree

Call a function f on every node of a tree, in depth-first prefix order

```
void dfs(const auto &tree, const auto &root)
{
    f(root);

    for (auto child : tree)
        dfs(tree, child);
}
```

```
def DFS(f, root):
    f(root)
    for child in root:
        DFS(f, child)
```

19 Reverse a list

Reverse the order of the elements of the list x . This may reverse "in-place" and destroy the original ordering.

```
std::reverse(begin(x), end(x));
```

```
x = reversed(x)
```

Alternative implementation:

```
y = x[::-1]
```

Alternative implementation:

```
x.reverse()
```

20 Return two values

Implement a function

search which looks for item **x** in a 2D matrix **m**. Return indices **i, j** of the matching cell.

Think of the most idiomatic way in the language to return the two values at the same time.

```
template<typename T, size_t len_x, size_t len_y>
std::pair<size_t, size_t> search (const T (&m)[len_x][len_y], const T &x) {
    for(size_t pos_x = 0; pos_x < len_x; ++pos_x) {
        for(size_t pos_y = 0; pos_y < len_y; ++pos_y) {
            if(m[pos_x][pos_y] == x) {
                return std::pair<size_t, size_t>(pos_x, pos_y);
            }
        }
    }

    // return an invalid value if not found
    return std::pair<size_t, size_t>(len_x, len_y);
}
```

```
def search(m, x):
    for idx, item in enumerate(m):
        if x in item:
            return idx, item.index(x)
```

Alternative implementation:

```
bool _search(const Matrix& _m, const float _x, size_t* const out_i, size_t* const out_j) {
    for (size_t j = 0; j < _m.rows; j++)
    {
        for (size_t i = 0; i < _m.cols; i++)
        {
            if (_m.at(i, j) == _x) {
                *out_i = i;
                *out_j = j;
                return true;
            }
        }
    }
    return false;
}
```

21 Swap values

Swap the values of the variables **a** and **b**

```
swap(a, b);
```

```
a, b = b, a
```

Alternative implementation:

```
a =int(input("enter a number"))
b =int(input("enter b number"))
a, b = b, a

print("Value of a:", a)
print("Value of a", b)
```

22 Convert string to integer

Extract the integer value i from its string representation s (in radix 10)

```
int i = std::atoi(s);
```

```
i = int(s)
```

Alternative implementation:

```
int i = std::stoi(s);
```

Alternative implementation:

```
std::stringstream ss(str);
int i;
ss >> i;
```

Alternative implementation:

```
std::string s("123");
int i;
std::from_chars(s.data(), s.data() + s.size(), i, 10);
```

23 Convert real number to string with 2 decimal places

Given a real number x , create its string representation s with 2 decimal digits following the dot.

```
std::stringstream ss;
ss << std::fixed << std::setprecision(2) << x;
s = ss.str();
```

```
s = '{:.2f}'.format(x)
```

Alternative implementation:

```
s = f'{x:.2f}'
```

24 Assign to string the japanese word ネコ

Declare a new string s and initialize it with the literal value "ネコ" (which means "cat" in japanese)

```
std::string s = u8"ネコ";
```

```
s = "ネコ"
```

25 Send a value to another thread

Share the string value "Alan" with an existing running process which will then display "Hello, Alan"

```
//declaration
auto mutex = std::mutex{};
auto cv = std::condition_variable{};
auto variable = std::optional<std::string>{};
auto future = std::async([&]() {
    auto lock = std::unique_lock{mutex, std::defer_lock};
    cv.wait(lock, [&variable]() { return variable.has_value(); });
    std::cout << "Hello, " << *variable << "!" << std::endl;
});
```

```
//passing value in main thread (or it can be done in others as well)
{
    auto lock = std::unique_lock{mutex};
    variable = "Alan";
    cv.notify_all();
}
```

```
q = Queue()
```

```
def worker():
    while True:
        print(f"Hello, {q.get()}")
        q.task_done()
```

```
Thread(target=worker, daemon=True).start()
```

```
q.put("Alan")
q.join()
```

26 Create a 2-dimensional array

Declare and initialize a matrix x having m rows and n columns, containing real numbers.

```
std::vector<std::vector<double>> x (m, std::vector<double>(n));
```

```
x = [[0] * n for _ in range(m)]
```

27 Create a 3-dimensional array

Declare and initialize a 3D array **x**, having dimensions boundaries **m**, **n**, **p**, and containing real numbers.

```
std::vector<std::vector<std::vector<double>>> x (m, std::vector<std::vector<double>> (n, std::vector<double> (p)));
```

```
x = [[[0 for k in range(p)] for j in range(n)] for i in range(m)]
```

Alternative implementation:

```
x = numpy.zeros((m,n,p))
```

28 Sort by a property

Sort the elements of the list (or array-like collection) **items** in ascending order of **x.p**, where **p** is a field of the type **Item** of the objects in **items**.

```
std::sort(begin(items), end(items), [](const auto& a, const auto& b) { return a.p < b.p; });
```

```
items = sorted(items, key=lambda x: x.p)
```

Alternative implementation:

```
items = sorted(items, key=attrgetter('p'))
```

Alternative implementation:

```
struct {
    bool operator()(const Item &lhs, const Item &rhs) const { return lhs.p < rhs.p; }
} custom_sort;

std::sort(begin(items), end(items), custom_sort);
```

29 Remove item from list, by its index

Remove **i**-th item from list **items**.
This will alter the original list or return a new list, depending on which is more idiomatic.
Note that in most languages, the smallest valid value for **i** is **0**.

```
items.erase (items.begin () + i);
```

```
del items[i]
```

Alternative implementation:

```
items.pop(i)
```

30 Parallelize execution of 1000 independent tasks

Launch the concurrent execution of procedure **f** with parameter **i** from 1 to 1000.
Tasks are independent and **f(i)** doesn't return any value.
Tasks need not run all at the same time, so you may use a pool.

```
auto futures = std::vector<std::future<void>>{};
for (auto i = 1; i <= 1000; ++i)
{
    futures.emplace_back(std::async(f, i));
}
```

```
pool = Pool()
for i in range(1, 1001):
    pool.apply_async(f, [i])
```

31 Recursive factorial (simple)

Create the recursive function **f** which returns the factorial of the non-negative integer **i**, calculated from **f(i-1)**

```
unsigned int f( unsigned int i ) {
    if ( i == 0 ) return 1;

    return i * f( i - 1 );
}
```

```
def f(i):
    if i == 0:
        return 1
    else:
        return i * f(i-1)
```

32 Integer exponentiation by squaring

Create function **exp** which calculates (fast) the value **x** power **n**.
x and **n** are non-negative integers.

```
int pow(int x, int p)
{
    if (p == 0) return 1;
    if (p == 1) return x;

    int tmp = pow(x, p/2);
    if (p%2 == 0) return tmp * tmp;
    else return x * tmp * tmp;
}
```

```
def exp(x, n):
    return x**n
```

33 Atomically read and update variable

Assign to the variable x the new value $f(x)$, making sure that no other thread may modify x between the read and the write.

```
std::atomic<int> x{};

auto local_x = x.load();
while(!x.compare_exchange_strong(local_x, f(local_x))) {
    local_x = x.load();
}
```

Alternative implementation:

```
//declaration
auto mutex = std::mutex{};
auto x = someValue();

//use
{
    auto lock = std::unique_lock{mutex};
    x = f(x);
}
```

```
lock = threading.Lock()

lock.acquire()
try:
    x = f(x)
finally:
    lock.release()
```

Alternative implementation:

```
with threading.Lock():
    x = f(x)
```

34 Create a set of objects

Declare and initialize a set x containing unique objects of type T .

```
std::unordered_set<T> x;
```

Alternative implementation:

```
std::unordered_set<T, hasher, eq> x;
```

```
class T(object):
    pass
```

```
x = set(T())
```

Alternative implementation:

```
class T:
    ...

s = set(T() for _ in range(x))
```

35 First-class function : compose

Implement a function **compose** ($A \rightarrow C$) with parameters f ($A \rightarrow B$) and g ($B \rightarrow C$), which returns the composition function $g \circ f$

```
std::function<C (A)> compose(B (&f)(A),
C (&g)(B))
{
    return [&f, &g](A a){return g(f(a));};
}

auto fn = compose(f, g);
```

```
def compose(f, g):
    return lambda a: g(f(a))
```

36 First-class function : generic composition

Implement a function **compose** which returns composition function $g \circ f$ for any functions f and g having exactly 1 parameter.

```
template<typename Pred1, typename Pred2>
>
auto compose(Pred1 f, Pred2 g){
    return [f,g](auto x){ return g(f(x));
};
}
```

```
def compose(f, g):
    return lambda x: g(f(x))
```

37 Currying

Transform a function that takes multiple arguments into a function for which some of the arguments are preset.

```
// function taking many parameters
int add(int a, int b)
{
    return a + b;
}

// define a new function presetting the
// first parameter
std::function<int (int)> add_def(int a)
{
    return [a](int b){return add(a,
b);};
}

int result = add_def(4)(6);
```

```
def add(a, b):
    return a+b

add_to_two = partial(add, 2)
```

Alternative implementation:

```
//function
auto add(int a, int b) -> int {
    return a + b;
}

//curry with std::bind
using namespace std::placeholders;
auto add5 = std::bind(add, _1, 5);

//curry with lambda
auto add5 = [](int x) { return add(x,
5); };

//use
auto result = add5(1);
assert(result == 6);
```

38 Extract a substring

Find substring **t** consisting in characters **i** (included) to **j** (excluded) of string **s**. Character indices start at **0** unless specified otherwise. Make sure that multibyte characters are properly handled.

```
auto t = s.substr(i, j-i);
```

```
t = s[i:j]
```

39 Check if string contains a word

Set the boolean **ok** to **true** if the string **word** is contained in string **s** as a substring, or to **false** otherwise.

```
bool ok = s.find(word) != std::string::
npos;
```

```
ok = word in s
```

41 Reverse a string

Create string **t** containing the same characters as string **s**, in reverse order. Original string **s** must remain unaltered. Each character must be handled correctly regardless its number of bytes in memory.

```
std::string s("Example");
std::string t;
std::copy(s.crbegin(), s.crend(), std::
back_inserter(t));
```

```
t = s.decode('utf8')[::-1].encode('utf
8')
```

Alternative implementation:

```
t = s[::-1]
```


42 Continue outer loop

Print each item **v** of list **a** which is not contained in list **b**.

For this, write an outer loop to iterate on **a** and an inner loop to iterate on **b**.

```
auto a = {1,2,3,4,5};
auto b = {3,5};

for (auto va: a){
    for (auto vb: b){
        if (va==vb) goto OUTER;
    }
    std::cout << va << '\n';
    OUTER: continue;
}
```

```
for v in a:
    try:
        for u in b:
            if v == u:
                raise Exception()
        print(v)
    except Exception:
        continue
```

Alternative implementation:

```
for v in a:
    keep = True
    for w in b:
        if w == v:
            keep = False
            break
    if keep:
        print(v)
```

43 Break outer loop

Look for a negative value **v** in 2D integer matrix **m**. Print it and stop searching.

```
auto indices = findNegativeValue (m, 1
0, 20);
std::cout << m[indices.first][indices.s
econd] << '\n';

std::pair<int, int> findNegativeValue
(int **m, int rows, int columns) {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            if (m[i][j] < 0) return make_pair
(i, j);
        }
    }
    throw "No negative value!";
}
```

```
class BreakOuterLoop (Exception): pass

try:
    position = None
    for row in m:
        for column in m[row]:
            if m[row][column] == v:
                position = (row, column)
            raise BreakOuterLoop
except BreakOuterLoop:
    pass
```

Alternative implementation:

```
def loop_breaking(m, v):
    for i, row in enumerate(m):
        for j, value in enumerate(row):
            if value == v:
                return (i, j)
    return None

print(loop_breaking([1,2,3],[4,5,6],
[7,8,9]), 6))
```

Alternative implementation:

```
matrix = [[1,2,3],[4,-5,6],[7,8,9]]
try:
    print(next(i for i in chain.from_ite
erable(matrix) if i < 0))
except StopIteration:
    pass
```

44 Insert element in list

Insert the element **x** at position **i** in the list **s**. Further elements must be shifted to the right.

```
s.insert (s.begin () + i, x);
```

```
s.insert(i, x)
```

45 Pause execution for 5 seconds

Sleep for 5 seconds in current thread, before proceeding with the next instructions.

```
std::this_thread::sleep_for(5s);
```

```
time.sleep(5)
```

46 Extract beginning of string (prefix)

Create the string **t** consisting of the 5 first characters of the string **s**.
Make sure that multibyte characters are properly handled.

```
auto t = s.substr(0, 5);
```

```
t = s[:5]
```

47 Extract string suffix

Create string **t** consisting in the 5 last characters of string **s**.
Make sure that multibyte characters are properly handled.

```
std::string t = s.substr(s.length() - 5);
```

```
t = s[-5:]
```

48 Multi-line string literal

Assign to variable **s** a string literal consisting in several lines of text, including newlines.

```
std::string s = R"( Earth is a planet.
So is the Jupiter)";
```

```
s = """Huey
Dewey
Louie"""
```

49 Split a space-separated string

Build list **chunks** consisting in substrings of the string **s**, separated by one or more space characters.

```
std::istringstream x(s);
std::list<std::string> chunks;
std::copy(std::istream_iterator<std::string>(x), std::istream_iterator<std::string>(), std::back_inserter(chunks));
```

```
chunks = s.split()
```

50 Make an infinite loop

Write a loop that has no end clause.

```
for (;;) {
    /// Do something
}
```

```
while True:
    pass
```

Alternative implementation:

```
while (true) {
    // Do something
}
```

51 Check if map contains key

Determine whether the map **m** contains an entry for the key **k**

```
bool key_exists = m.find(k) != m.end();
```

```
k in m
```

Alternative implementation:

```
bool key_exists = m.count(k) != 0;
```

Alternative implementation:

```
bool key_exists = m.contains(k);
```

52 Check if map contains value

Determine whether the map **m** contains an entry with the value **v**, for some key.

```
std::find_if(m.begin(), m.end(), [v](const auto& mo) {return mo.second == v;
}) != m.end();
```

```
v in m.values()
```

53 Join a list of strings

Concatenate elements of string list **x** joined by the separator **"**, **"** to create a single string **y**.

```
std::vector<std::string> x;
std::string y;
const char* const delim = ", ";

switch (x.size())
{
    case 0: y = ""; break;
    case 1: y = x[0]; break;
    default:
        std::ostringstream os;
        std::copy(x.begin(), x.
end() - 1,
                std::ostream_it
erator<std::string>(os, delim));
        os << *x.rbegin();
        y = os.str();
}
```

Alternative implementation:

```
y = x | views::join_with(',') | to<st
d::string>();
```

```
y = ', '.join(x)
```

Alternative implementation:

```
y = ', '.join(map(str, x))
```

54 Compute sum of integers

Calculate the sum **s** of the integer list or array **x**.

```
int s = std::accumulate(x.begin(), x.en
d(), 0, std::plus<int>());
```

```
s = sum(x)
```

55 Convert integer to string

Create the string representation **s** (in radix 10) of the integer value **i**.

```
auto s = std::to_string(i);
```

```
s = str(i)
```

57 Filter list

Create the list **y** containing the items from the list **x** that satisfy the predicate **p**. Respect the original ordering. Don't modify **x** in-place.

```
std::copy_if (x.begin (), x.end (), st
d::back_inserter(y), p);
```

```
y = list(filter(p, x))
```

Alternative implementation:

```
y = [element for element in x if p(elem
ent)]
```

58 Extract file content to a string

Create the string **lines** from the content of the file with filename **f**.

```
std::string fromFile(std::string _f)
{
    std::ifstream t(_f);
    t.seekg(0, std::ios::end);
    size_t size = t.tellg();
    std::string buffer(size, ' ');
    t.seekg(0);
    t.read(&buffer[0], size);
}
```

```
lines = open(f).read()
```

Alternative implementation:

```
with open(f) as fo:
    lines = fo.read()
```

59 Write to standard error stream

Print the message "**x** is negative" to standard error (stderr), with integer **x** value substitution (e.g. "-2 is negative").

```
std::cerr << x << " is negative\n";
```

Alternative implementation:

```
int main(){
    int x = -2;
    std::cerr << x <<" is negative
\n";
}
```

```
print(x, "is negative", file=sys.stder
r)
```

60 Read command line argument

Assign to **x** the string value of the first command line parameter, after the program name.

```
int main(int argc, char *argv[])
{
    vector<string> args(1 + argc, argc + a
rgv);

    string x = args.at(0);
}
```

```
x = sys.argv[1]
```

61 Get current date

Assign to the variable **d** the current date/time value, in the most standard type.

```
std::time_t d = std::time(nullptr);
```

```
d = datetime.datetime.now()
```

62 Find substring position

Set **i** to the first position of string **y** inside string **x**, if exists.

```
int i = x.find(y);
```

```
i = x.find(y)
```

Specify if **i** should be regarded as a **character** index or as a **byte** index.

Explain the behavior when **y** is not contained in **x**.

63 Replace fragment of a string

Assign to **x2** the value of string **x** with all occurrences of **y** replaced by **z**.

Assume occurrences of **y** are not overlapping.

```
std::stringstream s;
std::regex_replace(std::ostreambuf_iter
ator<char>(s), x.begin(), x.end(), y,
z);
auto x2 = s.str()
```

```
x2 = x.replace(y, z)
```

68 Create a bitset

Create an object **x** to store **n** bits (**n** being potentially large).

```
std::vector<bool> x(n, 0);
```

```
x = bytearray(int(math.ceil(n / 8.0)))
```

Alternative implementation:

```
std::bitset<n> x;
```

69 Seed random generator

Use seed **s** to initialize a random generator.

```
srand(s);
```

```
rand = random.Random(s)
```

If **s** is constant, the generator output will be the same each time the program runs. If **s** is based on the current value of the system clock, the generator output will be different each time.

71 Echo program implementation

Basic implementation of the Echo program: Print all arguments except the program name, separated by space, followed by newline. The idiom demonstrates how to skip the first argument if necessary, concatenate arguments as strings, append newline and print it to stdout.

```
int main(int argc, char * argv[])
{
    for(int i = 1; i < argc; ++i)
    {
        std::cout <<(1==i ? ""
: " ") << argv[i];
    }
    std::cout <<std::endl;
}
```

```
print(' '.join(sys.argv[1:]))
```

73 Create a factory

Create a factory named **fact** for any sub class of **Parent** and taking exactly one string **str** as constructor parameter.

```
template <typename T,
          std::enable_if_t<
            std::is_base_of_v<Parent, T
>, bool> = true>
T fact(const std::string& str) {
    return T(str);
}
```

```
def fact(a_class, str_):
    if issubclass(a_class, Parent):
        return a_class(str_)
```

74 Compute GCD

Compute the greatest common divisor **x** of big integers **a** and **b**. Use an integer type able to handle huge numbers.

```
unsigned long long int GCD(unsigned long long int a, unsigned long long int b)
{
    unsigned long long int c=a%b;
    if(c==0)
        return b;
    return GCD(b, c);
}
```

```
x = gcd(a, b)
```

Alternative implementation:

```
x = math.gcd(a, b)
```

Alternative implementation:

```
auto x = std::gcd(a, b);
```

75 Compute LCM

Compute the least common multiple **x** of big integers **a** and **b**. Use an integer type able to handle huge numbers.

```
auto x = std::lcm(a, b);
```

```
x = (a*b)//gcd(a, b)
```

Alternative implementation:

```
x = math.lcm(a, b)
```

76 Binary digits from an integer

Create the string **s** of integer **x** written in base 2.

```
std::bitset<sizeof(x)*8> y(x);
auto s = y.to_string();
```

```
s = '{:b}'.format(x)
```

E.g. 13 -> "1101"

77 Complex number

Declare a complex **x** and initialize it with value (**3i** - **2**). Then multiply it by **i**.

```
using namespace std::complex_literals;

auto x = 3i - 2.;
x *= 1i;
```

```
x = 3j-2
y = x * 1j
```

78 "do while" loop

Execute a block once, then execute it again as long as boolean condition **c** is true.

```
do {
    something();
    someOtherThing();
} while(c);
```

```
while True:
    do_something()
    if not c:
        break
```

79 Convert integer to floating point number

Declare the floating point number **y** and initialize it with the value of the integer **x**.

```
float y = static_cast<float>(x);
```

```
y = float(x)
```

80 Truncate floating point number to integer

Declare integer **y** and initialize it with the value of floating point number **x**. Ignore non-integer digits of **x**. Make sure to truncate towards zero: a negative **x** must yield the closest greater integer (not lesser).

```
int y = static_cast<int>(x);
```

```
y = int(x)
```

81 Round floating point number to integer

Declare the integer **y** and initialize it with the rounded value of the floating point number **x**. Ties (when the fractional part of **x** is exactly .5) must be rounded up (to positive infinity).

```
int y = static_cast<int>(std::floor(x + 0.5f));
```

```
y = int(x + 0.5)
```

83 Regex with character repetition

Declare regular expression **r** matching strings "http", "http", "http", etc.

```
auto r = regex("htt+p");
```

```
r = re.compile(r"htt+p")
```

84 Count bits set in integer binary representation

Count number **c** of 1s in the integer **i** in base 2.

```
uint32_t c = 0;
for (; i; i &= i - 1, ++c);
```

```
c = bin(i).count("1")
```

Alternative implementation:

```
c = i.bit_count()
```

E.g. $i=6 \rightarrow c=2$

85 Check if integer addition will overflow

Write boolean function **addingWillOverflow** which takes two integers **x, y** and return **true** if **(x+y)** overflows.

```
bool addingWillOverflow(int x, int y) {
    return ((x > 0) && (y > std::numeric_
limits<int>::max() - x)) ||
           ((x < 0) && (y < std::numeric_
limits<int>::min() - x));
}
```

```
def adding_will_overflow(x,y):
    return False
```

An overflow may be **above** the max positive value, or **below** the min negative value.

87 Stop program

Exit immediately. If some extra cleanup work is executed by the program runtime (not by the OS itself), describe it.

```
exit(1);
```

```
sys.exit(1)
```

88 Allocate 1M bytes

Create a new bytes buffer **buf** of size 1,000,000.

```
vector<byte> _buf(1'000'000);
```

```
buf = bytearray(1000000)
```

89 Handle invalid argument

You've detected that the integer value of argument **x** passed to the current function is invalid. Write the idiomatic way to abort the function execution and signal the problem.

```
throw domain_error("oops!");
```

```
raise ValueError("x is invalid")
```

90 Read-only outside

Expose a read-only integer **x** to the outside world while being writable inside a structure or a class **Foo**.

```
class Foo final {
    int mX = 0;

public:
    const auto& X() const { return mX; }
};
```

```
class Foo(object):
    def __init__(self):
        self._x = 0

    @property
    def x(self):
        """
        Doc for x
        """
        return self._x
```

93 Pass a runnable procedure as parameter

Implement the procedure **control** which receives one parameter **f**, and runs **f**.

```
void control(invocable auto&& f)
{
    f();
}
```

```
def control(f):
    f()
```

94 Print the type of a variable

Print the name of the type of **x**. Explain if it is a static type or dynamic type.

```
std::cout<<typeid(x).name();
```

```
print(type(x))
```

Alternative implementation:

```
print(x.__class__)
```

This may not make sense in all languages.

95 Get file size

Assign to variable **x** the length (number of bytes) of the local file at **path**.

```
std::ifstream f(path, std::ios::in | std::ios::binary);
f.seekg(0, std::ios::end);
size_t x = f.tellg();
```

```
x = os.path.getsize(path)
```

96 Check string prefix

Set the boolean **b** to **true** if string **s** starts with prefix **prefix**, **false** otherwise.

```
bool b = s.starts_with(prefix);
```

```
b = s.startswith(prefix)
```

Alternative implementation:

```
std::string prefix = "something";
bool b = s.compare(0, prefix.size(), prefix) == 0;
```

97 Check string suffix

Set boolean **b** to **true** if string **s** ends with string **suffix**, **false** otherwise.

```
return suffix.size() == s.size() - s.rfind(suffix);
```

```
b = s.endswith(suffix)
```

Alternative implementation:

```
bool b = s.ends_with(suffix);
```

98 Epoch seconds to date object

Convert a timestamp **ts** (number of seconds in epoch-time) to a date with time **d**. E.g. 0 -> 1970-01-01 00:00:00

```
std::time_t d = std::chrono::system_clock::to_time_t(ts);
```

```
d = datetime.date.fromtimestamp(ts)
```

99 Format date YYYY-MM-DD

Assign to the string **x** the value of the fields (year, month, day) of the date **d**, in format **YYYY-MM-DD**.

```
int main()
{
    char x[32]{};
    time_t a = time(nullptr);
    struct tm d;
    if (localtime_s(&d, &a) == 0) {
        strftime(x, sizeof(x),
"%F", &d);
        std::cout << x << std::endl;
    }
    return 0;
}
```

```
d = date(2016, 9, 28)
x = d.strftime('%Y-%m-%d')
```

Alternative implementation:

```
d = date.today()
x = d.isoformat()
```

100 Sort by a comparator

Sort elements of array-like collection **items**, using a comparator **c**.

```
struct is_less {
    bool operator () (int a, int b){
        return a < b;
    }
};

int main(){
    std::vector<int> items = {1337, 66
6, -666, 0, 0, 666, -666};
    std::sort(items.begin(), items.end
(), is_less());

    std::vector<int> expected = {-666,
-666, 0, 0, 666, 666, 1337};
    assert(items.size() == expected.siz
e());
    for (size_t i = 0; i < items.size
(); i++){
        assert(items[i] == expected
[i]);
    }
    return 0;
}
```

```
items.sort(key=c)
```

Alternative implementation:

```
items.sort(key=functools.cmp_to_key(c))
```

105 Current executable name

Assign to the string **s** the name of the currently executing program (but not its full path).

```
int main(int argc, char* argv[])
{
    std::cout << std::filesystem::path
(argv[0]).filename() << '\n';
}
```

```
s = sys.argv[0]
```

106 Get program working directory

Assign to string **dir** the path of the working directory.
(This is not necessarily the folder containing the executable itself)

```
dir = std::filesystem::current_path();
```

```
dir = os.getcwd()
```

109 Number of bytes of a type

Set **n** to the number of bytes of a variable **t** (of type **T**).

```
std::size_t n = sizeof(t);
```

```
n = pymppler.asizeof.asizeof(t)
```


110 Check if string is blank

Set the boolean **blank** to **true** if the string **s** is empty, or null, or contains only whitespace; **false** otherwise.

```
bool blank = false;
if (s.empty() || std::all_of(s.begin(),
s.end(), [](char c){return std::isspace
(c);})) {
    blank = true;
}
```

```
blank = not s or s.isspace()
```

112 Iterate over map entries, ordered by keys

Print each key **k** with its value **x** from an associative array **mymap**, in ascending order of **k**.

```
std::map< K, V > _mymap;
for (const auto& pair : _mymap) {
    std::cout << pair.first << ": " <<
pair.second << "\n";
}
```

```
for k in sorted(mymap):
    print(mymap[k])
```

Alternative implementation:

```
auto print(auto&... args) {
    // c++17 fold expression
    (std::cout << ... << args) << std::endl;
}

auto print_map_contents(auto mymap) {
    // c++17 structured binding
    for (auto [k, x] : mymap) {
        print("mymap[" , k, "] = ", x);
    }
}

auto main() -> int {
    // map is ordered map, iteration is a
    // scending by key value
    auto map = std::map<std::string, int>
    {
        {"first entry", 12},
        {"second entry", 42},
        {"third entry", 3},
    };

    print_map_contents(map);
}
```

113 Iterate over map entries, ordered by values

Print each key **k** with its value **x** from an associative array **mymap**, in ascending order of **x**. Multiple entries may exist for the same value **x**.

```
for_each(begin(mymap), end(mymap),
[&s](const auto& kv) { s.insert(kv.
second); });
```

```
for x, k in sorted((x, k) for k,x in my
map.items()):
    print(k, x)
```

Alternative implementation:

```
for key, value in sorted(d.items(), key
=operator.itemgetter(1)):
    print(key, value)
```

117 Get list size

Set **n** to the number of elements of the list **x**.

```
size_t n = x.size();
```

```
n = len(x)
```

118 List to set

Create the set **y** from the list **x**.

x may contain duplicates. **y** is unordered and has no repeated values.

```
std::unordered_set<T> y (x.begin (), x.
end ());
```

```
y = set(x)
```

Alternative implementation:

```
std::set<T> y (x.begin (), x.end ());
```

119 Deduplicate list

Remove duplicates from the list **x**.
Explain if the original order is preserved.

```
std::sort(x.begin(), x.end());
auto last = std::unique(x.begin(), x.end());
x.erase(last, x.end());
```

Alternative implementation:

```
std::vector<std::string> x = {"one", "two", "two", "one", "three"};
std::unordered_set<std::string> t;
for (auto e : x)
    t.insert(e);
```

```
x = list(set(x))
```

Alternative implementation:

```
x = list(OrderedDict(zip(x, x)))
```

Alternative implementation:

```
def dedup(x):
    y = []
    for i in x:
        if not i in y:
            y.append(i)
    return y
```

120 Read integer from stdin

Read an integer value from the standard input into the variable **n**

```
std::cin >> n;
```

```
n = int(input("Input Prompting String:"))
```

122 Declare an enumeration

Create an enumerated type **Suit** with 4 possible values **SPADES**, **HEARTS**, **DIAMONDS**, **CLUBS**.

```
enum class Suit {
    SPADES, HEARTS, DIAMONDS, CLUBS
};
```

Alternative implementation:

```
enum class Color : char{
    Red, Black, Green
};
```

```
class Suit:
    SPADES, HEARTS, DIAMONDS, CLUBS
    = range(4)
```

Alternative implementation:

```
class Suit(Enum):
    SPADES = 1
    HEARTS = 2
    DIAMONDS = 3
    CLUBS = 4
```

123 Assert condition

Verify that predicate **isConsistent** returns true, otherwise report assertion violation.
Explain if the assertion is executed even in production environment or not.

```
assert(isConsistent());
```

```
assert isConsistent
```

124 Binary search for a value in sorted array

Write the function **binarySearch** which returns the index of an element having the value **x** in the sorted array **a**, or -1 if no such element exists.

```
template<typename T>
int binarySearch(const std::vector<T> &
a, const T &x)
{
    if(a.size() == 0) return -1;

    size_t lower = 0;
    size_t upper = a.size() - 1;

    while(lower <= upper)
    {
        auto mid = lower + (upper-lower) / 2;

        if(x == a[mid])
        {
            return (int)mid;
        }
        else if(x > a[mid])
        {
            lower = mid + 1;
        }
        else
        {
            upper = mid - 1;
        }
    }

    return -1;
}
```

```
def binarySearch(a, x):
    i = bisect.bisect_left(a, x)
    return i if i != len(a) and a[i] == x else -1
```

125 Measure function call duration

measure the duration **t**, in nanoseconds, of a call to the function **foo**. Print this duration.

```
auto start = std::chrono::steady_clock::now();
foo();
auto end = std::chrono::steady_clock::now();
auto t = std::chrono::duration_cast<std::chrono::nanoseconds>(end-start).count();
```

```
t1 = time.perf_counter_ns()
foo()
t2 = time.perf_counter_ns()
print('Nanoseconds:', t2 - t1)
```

126 Multiple return values

Write a function **foo** that returns a **string** and a **boolean** value.

```
std::tuple<std::string, bool> foo() {
    return std::make_tuple("someString", true);
}
```

```
def foo():
    return 'string', True
```

Alternative implementation:

```
auto foo()
{
    return std::make_tuple("Hello", true);
}
```

127 Source code inclusion

Import the source code for the function **foo** body from a file "foobody.txt".

```
void _foo() {
#include "foobody.txt"
}
```

```
foo = imp.load_module('foobody', 'foobody.txt').foo
```

128 Breadth-first traversing of a tree

Call a function *f* on every node of a tree, in breadth-first prefix order

```
void bfs(Node& root, std::function<void
(Node*)> f) {
    std::deque<Node*> node_queue;
    node_queue.push_back(&root);
    while (!node_queue.empty()) {
        Node* const node = node_queue.front
        ();
        node_queue.pop_front();
        f(node);
        for (Node* const child : node->chil
        dren) {
            node_queue.push_back(child);
        }
    }
}
```

```
def BFS(f, root):
    Q = [root]
    while Q:
        n = Q.pop(0)
        f(n)
        for child in n:
            if not n.discov
            ered:
                n.disco
                vered = True
            Q.append(n)
```

130 Depth-first traversal in a graph

Call the function *f* on every vertex accessible from the vertex *v*, in depth-first prefix order

```
void dfs(Node& root, std::function<void
(Node*)> f) {
    std::stack<Node*> queue;
    queue.push(&root);
    std::unordered_set<const Node*> visit
    ed;
    while (!queue.empty()) {
        Node* const node = queue.top();
        queue.pop();
        f(node);
        visited.insert(node);
        for (Node* const neighbor : node->n
        eighbors) {
            if (visited.find(neighbor) == vis
            ited.end()) {
                queue.push(neighbor);
            }
        }
    }
}
```

```
def depth_first(start, f):
    seen = set()
    stack = [start]
    while stack:
        vertex = stack.pop()
        f(vertex)
        seen.add(vertex)
        stack.extend(
            v for v in vertex.adjacent if v n
            ot in seen
        )
```

131 Successive conditions

Execute *f1* if condition *c1* is true, or else *f2* if condition *c2* is true, or else *f3* if condition *c3* is true.
Don't evaluate a condition when a previous condition was true.

```
if (c1)
    f1();
else if (c2)
    f2();
else if (c3)
    f3();
```

```
f1() if c1 else f2() if c2 else f3() if
c3 else None
```

Alternative implementation:

```
if c1:
    f1()
elif c2:
    f2()
elif c3:
    f3()
```

Alternative implementation:

```
if (c1)
{
    f1();
}
else if (c2)
{
    f2();
}
else if (c3)
{
    f3();
}
```

Alternative implementation:

```
(c1 && (f1(), true))
|| (c2 && (f2(), true))
|| (c3 && (f3(), true));
```

132 Measure duration of procedure execution

Run the procedure *f*, and return the duration of the execution of *f*.

```
std::clock_t start = std::clock();
f();
double duration = (std::clock() - start) / (double) CLOCKS_PER_SEC;
```

Alternative implementation:

```
#include <chrono>

double measure() {
    using namespace std::chrono;
    const auto start{ high_resolution_clock::now() };
    f();
    const auto elapsed{ high_resolution_clock::now() - start };
    const double seconds{ duration_cast<duration<double>>(elapsed).count() };
    return seconds;
}
```

```
duration = timeit.timeit("f()", setup="from __main__ import f")
```

Alternative implementation:

```
start = time.time()
f()
end = time.time()
return end - start
```

133 Case-insensitive string contains

Set boolean *ok* to *true* if string *word* is contained in string *s* as a substring, even if the case doesn't match, or to *false* otherwise.

```
auto ok = std::search(std::begin(s), std::end(s), std::begin(word), std::end(word),
    [](auto c, auto d){
        return std::tolower(c) == std::tolower(d);
    }) != std::end(s);
```

```
ok = word.lower() in s.lower()
```

134 Create a new list

Declare and initialize a new list *items*, containing 3 elements *a*, *b*, *c*.

```
std::vector<T> items = {a, b, c};
```

Alternative implementation:

```
list items { a, b, c };
```

```
items = [a, b, c]
```

135 Remove item from list, by its value

Remove at most 1 item from list *items*, having the value *x*.

This will alter the original list or return a new list, depending on which is more idiomatic.

If there are several occurrences of *x* in *items*, remove only one of them. If *x* is absent, keep *items* unchanged.

```
erase(items, x);
```

```
items.remove(x)
```

136 Remove all occurrences of a value from a list

Remove all occurrences of the value *x* from list *items*.

This will alter the original list or return a new list, depending on which is more idiomatic.

```
items.remove(x);
```

```
newlist = [item for item in items if item != x]
```

137 Check if string**contains only digits**

Set the boolean ***b*** to ***true*** if the string ***s*** contains only characters in the range '0'..'9', ***false*** otherwise.

```
bool b = false;
if (! s.empty() && std::all_of(s.begin(), s.end(), [](char c){return std::isdigit(c);})) {
    b = true;
}
```

```
b = s.isdigit()
```

140 Delete map entry

Delete from map ***m*** the entry having key ***k***.

```
m.erase(k);
```

```
m.pop(k, None)
```

Explain what happens if ***k*** is not an existing key in ***m***.

141 Iterate in sequence over two lists

Iterate in sequence over the elements of the list ***items1*** then ***items2***. For each iteration print the element.

```
void print_seq(const auto& ...xs)
{
    (std::for_each(std::begin(xs), std::end(xs),
        [](const auto& x) { std::cout << x; }, ...));
}

std::list xs { "hi", "there", "world" }, ys { "lo" "thar" };

print_seq(xs, ys);
```

```
for x in items1 + items2:
    print(x)
```

142 Hexadecimal digits of an integer

Assign to string ***s*** the hexadecimal representation (base 16) of integer ***x***.

E.g. 999 -> "3e7"

```
template <typename I>
std::string n2hexstr(I w, size_t hex_len = sizeof(I)<<1) {
    static const char* digits = "0123456789ABCDEF";
    std::string str(hex_len, '-');
    for (size_t i=0, j=(hex_len-1)*4; i<hex_len; ++i,j-=4)
        str[i] = digits[(w>>j) & 0xf];
    return str;
}
```

```
s = hex(x)
```

Alternative implementation:

```
std::ostringstream stream;
stream << std::hex << x;
s = stream.str();
```

144 Check if file exists

Set boolean ***b*** to ***true*** if file at path ***fp*** exists on filesystem; ***false*** otherwise.

```
bool b = std::filesystem::exists(fp);
```

```
b = os.path.exists(fp)
```

Alternative implementation:

```
b = Path(fp).exists()
```

Beware that you should never do this and then in the next instruction assume the result is still valid, this is a race condition on any multitasking OS.

146 Convert string to floating point number

Extract floating point value *f* from its string representation *s*

```
float f = std::stof(s);
```

```
s = u'545,2222'
locale.setlocale(locale.LC_ALL, 'de')
f = locale.atof(s)
```

Alternative implementation:

```
f = float(s)
```

Alternative implementation:

```
f = float(s)
```

147 Remove all non-ASCII characters

Create string *t* from string *s*, keeping only ASCII characters

```
copy_if(begin(src), end(src), back_inserter(dest),
        [](const auto c) { return static_cast<unsigned char>(c) <= 0x7F; });
```

```
t = re.sub('[^\u0000-\u007f]', '', s)
```

Alternative implementation:

```
t = s.encode("ascii", "ignore").decode()
```

148 Read list of integers from stdin

Read a list of integer numbers from the standard input, until EOF.

```
vector<int> v;
for(int t;;){
    cin >> t;
    if(cin.eof())
        break;
    v.push_back(t);
}
```

```
list(map(int, input().split()))
```

Alternative implementation:

```
numbers = [int(x) for x in input().split()]
```

150 Remove trailing slash

Remove the last character from the string *p*, if this character is a forward slash /

```
if('/') == s.back()
    s.pop_back();
```

```
p = p.rstrip("/")
```

152 Turn a character into a string

Create string *s* containing only the character *c*.

```
string s { 'c' };
```

```
s = c
```

153 Concatenate string with integer

Create the string *t* as the concatenation of the string *s* and the integer *i*.

```
auto t = s + std::to_string(i);
```

```
t = f"{s}{i}"
```

155 Delete file

Delete from filesystem the file having path *filepath*.

```
auto r = std::filesystem::remove(filepath);
```

```
path = pathlib.Path(filepath)
path.unlink()
```

Alternative implementation:

```
os.remove(filepath)
```

157 Declare constant string

Initialize a constant *planet* with string value "Earth".

```
// using char*
constexpr char planet[] = "Earth";

// using string
const std::string planet{"Earth"};
```

```
PLANET = 'Earth'
```

160 Detect if 32-bit or 64-bit architecture

Execute `f32()` if platform is 32-bit, or `f64()` if platform is 64-bit. This can be either a compile-time condition (depending on target) or a runtime detection.

```
#ifdef __x86_64
f64();
#else
#ifdef _M_AMD64
f64();
#else
f32();
#endif
#endif
```

```
if sys.maxsize > 2**32:
    f64()
else:
    f32()
```

Alternative implementation:

```
if constexpr(sizeof(nullptr) == 8) {
    f64();
} else {
    f32();
}
```

161 Multiply all the elements of a list

Multiply all the elements of the list `elements` by a constant `c`

```
for (auto &it : elements)
    it *= c;
```

```
elements = [c * x for x in elements]
```

Alternative implementation:

```
std::transform(std::begin(elements), std::end(elements), std::begin(elements),
[c](auto i){
    return i*c;
});
```

165 Last element of list

Assign to the variable `x` the last element of the list `items`.

```
std::vector<int> items;
int last = items.back();
```

```
x = items[-1]
```

Alternative implementation:

```
auto x = *std::crbegin(items);
```

166 Concatenate two lists

Create the list `ab` containing all the elements of the list `a`, followed by all the elements of the list `b`.

```
auto ab = a;
ab.insert (ab.end (), b.begin (), b.end ());
```

```
ab = a + b
```

169 String length

Assign to the integer `n` the number of characters of the string `s`. Make sure that multibyte characters are properly handled. `n` can be different from the number of bytes of `s`.

```
auto utf8len(std::string_view const& s)
-> size_t
{
    std::setlocale(LC_ALL, "en_US.utf8");

    auto n = size_t{};
    auto size = size_t{};
    auto mb = std::mbstate_t{};

    while(size < s.length())
    {
        size += mbrlen(s.data()
+ size, s.length() - size, &mb);
        n += 1;
    }

    return n;
}
```

```
n = len(s)
```


170 Get map size

Set n to the number of elements stored in *mymap*.

This is not always equal to the map capacity.

```
auto n = mymap.size();
```

```
n = len(mymap)
```

171 Add an element at the end of a list

Append the element x to the list s .

```
s.emplace_back(x);
```

```
s.append(x)
```

Alternative implementation:

```
s.push_back(x);
```

172 Insert an entry in a map

Insert value v for key k in map m .

```
m[k] = v;
```

```
m[k] = v
```

179 Get center of a rectangle

Return the center c of the rectangle with coordinates $(x1, y1, x2, y2)$

```
template <typename T> struct Point {
    T x = {};
    T y = {};
};
template <typename T> struct Rectangle
{
    Point<T> upper_left;
    Point<T> lower_right;
};
template <typename T>
Point<T> get_center(const Rectangle<T>&
rect) {
    return {
        .x = (rect.upper_left.x + rect.lower_right.x) / 2,
        .y = (rect.upper_left.y + rect.lower_right.y) / 2,
    };
}
```

```
center = ((x1+x2)/2, (y1+y2)/2)
```

Alternative implementation:

```
Point = namedtuple('Point', 'x y')
center = Point((x1+x2)/2, (y1+y2)/2)
```

180 List files in directory

Create the list x containing the contents of the directory d .

x may contain files and subfolders.
No recursive subfolder listing.

```
auto directory_contents(auto path) {
    auto iterator = std::filesystem::directory_iterator(path);
    return std::vector<std::filesystem::path>(
        std::ranges::begin(iterator),
        std::ranges::end(iterator)
    );
}

auto main(int argc, char** argv) -> int
{
    auto path = argc >= 2
        ? std::filesystem::path(argv[1])
        : std::filesystem::current_path();

    for (auto entry : directory_contents(path)) {
        std::cout << entry.string() << std::endl;
    }
}
```

```
x = os.listdir(d)
```

189 Filter and transform list

Produce a new list **y** containing the result of the function **T** applied to all elements **e** of the list **x** that match the predicate **P**.

```
template <typename SeqT>
auto transform_copy_if(const SeqT i, auto p, auto f)
{
    using namespace std;

    SeqT o;

    for_each(begin(i), end(i),
        [&](const auto& x) {
            if(p(x))
                o.push_back(f(x));
        });

    return o;
}
```

```
y = [T(e) for e in x if P(e)]
```

Alternative implementation:

```
y = list(map(T, filter(P, x)))
```

Alternative implementation:

```
constexpr auto transform_matches(const
auto& x, auto p, auto t) {
    auto result = x
        | std::views::filter(p)
        | std::views::transform(t);

    std::vector<std::ranges::range_value_t<decltype(result)>> y;
    std::copy(result.begin(), result.end(), std::back_inserter(y));
    return y;
}
```

190 Call an external C function

Declare an external C function with the prototype

```
void foo(double *a, int n);
```

and call it, passing an array (or a list) of size 10 to **a** and 10 to **n**.

Use only standard features of your language.

```
extern "C" void foo(double *a, int n);
```

```
a = (c_int * 10)(*range(10))
n = 10
libc = cdll.LoadLibrary('/path/to/c_library')
libc.foo(c_void_p(a), c_int(n))
```

191 Check if any value in a list is larger than a limit

Given a one-dimensional array **a**, check if any value is larger than **x**, and execute the procedure **f** if that is the case

```
if (std::any_of(a.begin(), a.end(), [x]
(auto y) { return y > x; }))
    f();
```

```
if any(v > x for v in a):
    f()
```

197 Get a list of lines from a file

Retrieve the contents of file at **path** into a list of strings **lines**, in which each element is a line of the file.

```
std::ifstream file (path);
for (std::string line; std::getline(file, line); lines.push_back(line)) {}
```

```
with open(path) as f:
    lines = f.readlines()
```

200 Return hypotenuse

Returns the hypotenuse ***h*** of the triangle where the sides adjacent to the square angle have lengths ***x*** and ***y***.

```
auto h = std::hypot(x, y);
```

```
h = math.hypot(x, y)
```

205 Get an environment variable

Read an environment variable with the name "FOO" and assign it to the string variable ***foo***. If it does not exist or if the system does not support environment variables, assign a value of "none".

```
const char* tmp = std::getenv("FOO");
std::string foo = tmp ? std::string(tmp) : "none";
```

```
try:
    foo = os.environ['FOO']
except KeyError:
    foo = "none"
```

Alternative implementation:

```
foo = getenv('FOO', 'none')
```

Alternative implementation:

```
foo = os.environ.get('FOO', 'none')
```

209 Type with automatic deep deallocation

Declare a type ***t*** which contains a string ***s*** and an integer array ***n*** with variable size, and allocate a variable ***v*** of type ***t***. Allocate ***v.s*** and ***v.n*** and set them to the values "Hello, world!" for ***s*** and [1,4,9,16,25], respectively. Deallocate ***v***, automatically deallocating ***v.s*** and ***v.n*** (no memory leaks).

```
#include <string>
using namespace std::string_literals;
#include <vector>
#include <memory>

struct t {
    std::string s;
    std::vector<int> n;
};

auto v = std::make_unique<t>(
    "Hello, world!"s,
    decltype(t::n){1, 4, 9, 16, 25}
);
v.reset();

// Automatically:
void fn(){
    auto v = std::make_unique<t>(
        "Hello, world!"s,
        decltype(t::n){1, 4, 9, 16, 25}
    );
}
```

```
class T:
    def __init__(self, s, n):
        self.s = s
        self.n = n
        return

v = T('hello world', [1, 4, 9, 16, 25])
del v
```

211 Create folder

Create the folder at ***path*** on the filesystem

```
namespace fs = std::filesystem;
fs::create_directory(path);
```

```
os.mkdir(path)
```

212 Check if folder exists

Set the boolean ***b*** to ***true*** if ***path*** exists on the filesystem and is a directory; ***false*** otherwise.

```
auto b = std::filesystem::is_directory
(path);
```

```
b = os.path.isdir(path)
```

220 Create a tuple value

Create ***t*** consisting of 3 values having different types.

```
std::tuple<float, std::string, bool> t
(2.5f, "foo", false);
```

```
t = (2.5, "hello", -1)
```

Alternative implementation:

```
auto t = std::make_tuple(2.5f, std::string("foo"), false);
```

Explain if the elements of ***t*** are strongly typed or not.

222 Find the first index of an element in list

Set *i* to the first index in list *items* at which the element *x* can be found, or *-1* if *items* does not contain *x*.

```
auto it = std::find(items.begin(), items.end(), x);
i = it == items.end() ? std::distance(items.begin(), it) : -1;
```

```
i = items.index(x) if x in items else -1
```

223 for else loop

Loop through list *items* checking a condition. Do something else if no matches are found.

A typical use case is looping through a series of containers looking for one that matches a condition. If found, an item is inserted; otherwise, a new container is created.

These are mostly used as an inner nested loop, and in a location where refactoring inner logic into a separate function reduces clarity.

```
bool found = false;
for (const auto item : items) {
    if (item == "baz") {
        baz();
        found = true;
        break;
    }
}
if (!found) {
    DoSomethingElse();
}
```

Alternative implementation:

```
if (std::any_of(items.begin(), items.end(), condition))
    baz();
else
    DoSomethingElse();
```

```
for item in items:
    if item == 'baz':
        print('found it')
        break
else:
    print('never found it')
```

224 Add element to the beginning of the list

Insert the element *x* at the beginning of the list *items*.

```
items.emplace_front(x);
```

Alternative implementation:

```
items.push_front(x);
```

```
items = [x] + items
```

Alternative implementation:

```
items.insert(0, x)
```

225 Declare and use an optional argument

Declare an optional integer argument *x* to procedure *f*, printing out "Present" and its value if it is present, "Not present" otherwise

```
void f(std::optional<int> x = {}) {
    std::cout << (x ? "Present" + std::to_string(x.value()) : "Not Present");
}
```

Alternative implementation:

```
void f(std::optional<int> x = {}) {
    if (x) {
        std::cout << "Present" << x.value();
    } else {
        std::cout << "Not present";
    }
}
```

```
def f(x=None):
    if x is None:
        print("Not present")
    else:
        print("Present", x)
```

226 Delete last element from list

Remove the last element from the list *items*.

```
items.pop_back();
```

```
items.pop()
```

237 Xor integers

Assign to *c* the result of (*a xor b*)

```
int c = a ^ b;
```

```
c = a ^ b
```

239 Find first regular expression match

Assign to string **x** the first word of string **s** consisting of exactly 3 digits, or the empty string if no such match exists.

A word containing more digits, or 3 digits as a substring fragment, must not match.

```
std::regex re("\\b\\d{3}\\b");
auto it = std::sregex_iterator(s.begin(), s.end(), re);
std::string x = it == std::sregex_iterator() ? "" : it->str();
```

```
m = re.search(r'\b\d\d\d\b', s)
x = m.group(0) if m else ''
```

242 Iterate over a set

Call a function **f** on each element **e** of a set **x**.

```
std::for_each(x.begin(), x.end(), f);
```

```
for e in x:
    f(e)
```

Alternative implementation:

```
for (auto e : x)
    f(e);
```

Alternative implementation:

```
list(map(lambda e: f(e), x))
```

243 Print list

Print the contents of the list or array **a** on the standard output.

```
std::copy(a.begin(), a.end(), std::ostream_iterator<int>(std::cout, "\n"));
```

```
print(a)
```

244 Print a map

Print the contents of the map **m** to the standard output: keys and values.

```
for (auto entry : m) {
    std::cout << entry.first << ": "
    << entry.second << "\n";
}
```

```
print(m)
```

245 Print value of custom type

Print the value of object **x** having custom type **T**, for log or debug.

```
std::cout << x;
```

```
print(x)
```

247 Filter list in-place

Remove all the elements from list **x** that *don't* satisfy the predicate **p**, without allocating a new list.
Keep all the elements that *do* satisfy **p**.

```
std::list<Foo> x;
x.remove_if(std::not_fn(p));
```

```
del_count = 0
for i in range(len(x)):
    if not p(x[i - del_count]):
        del x[i - del_count]
    del_count += 1
```

Alternative implementation:

```
std::erase_if(x, std::not_fn(p));
```

For languages that don't have mutable lists, refer to idiom #57 instead.

251 Parse binary digits

Extract integer value **i** from its binary string representation **s** (in radix 2)
E.g. "1101" -> 13

```
int i = std::stoi(s, nullptr, 2);
```

```
i = int(s, 2)
```

252 Conditional assignment

Assign to the variable **x** the string value "a" if calling the function **condition** returns **true**, or the value "b" otherwise.

```
x = condition() ? "a" : "b";
```

```
x = "a" if condition() else "b"
```

254 Replace value in list

Replace all exact occurrences of "foo" with "bar" in the string list *x*

```
std::replace(x.begin(), x.end(), "foo",
"bar");
```

```
for i, v in enumerate(x):
    if v == "foo":
        x[i] = "bar"
```

Alternative implementation:

```
x = ["bar" if v=="foo" else v for v in
x]
```

256 Count backwards

Print the numbers 5, 4, ..., 0 (included), one line per number.

```
for (int i = 5; i >= 0; --i) {
    std::cout << i;
}
```

```
for i in range(5, -1, -1):
    print(i)
```

260 Create an empty list of strings

Declare a new list *items* of string elements, containing zero elements

```
std::vector<std::string> items;
```

```
items = []
```

271 Test for type extension

If a variable *x* passed to procedure *tst* is of type *foo*, print "Same type." If it is of a type that extends *foo*, print "Extends type." If it is neither, print "Not related."

```
template<typename T>
void tst(T)
{
    using namespace std;

    // Strip away any references and po
    inter
    typedef remove_const<remove_pointer
<decay<T>::type>::type>::type D;

    if(is_same<D, foo>::value)
    {
        cout << "same type" << endl;
    }
    else if(is_base_of<foo, D>::value)
    {
        cout << "extends type" << endl;
    }
    else
    {
        cout << "not related" << endl;
    }
}
```

```
def tst(x):
    if type(x) == foo:
        print("Same type.")
    elif isinstance(x, foo):
        print("Extends type.")
    else:
        print("Not related.")
```

272 Play FizzBuzz

Fizz buzz is a children's counting game, and a trivial programming task used to affirm that a programmer knows the basics of a language: loops, conditions and I/O.

The typical fizz buzz game is to count from 1 to 100, saying each number in turn. When the number is divisible by 3, instead say "Fizz". When the number is divisible by 5, instead say "Buzz". When the number is divisible by both 3 and 5, say "FizzBuzz".

```
for(int i = 1; i <= 100; i++)
{
    if((i % 15) == 0) std::cout << "FizzBuzz" << std::endl;
    else if((i % 5) == 0) std::cout << "Buzz" << std::endl;
    else if((i % 3) == 0) std::cout << "Fizz" << std::endl;
    else std::cout << i << std::endl;
}
```

Alternative implementation:

```
for (int n=1; n<=100; n++) {
    string out="";
    if (n%3==0) {
        out=out+"Fizz";
    }
    if (n%5==0) {
        out=out+"Buzz";
    }
    if (out=="") {
        out=out+std::to_string(n);
    }
    cout << out << "\n";
}
```

```
for i in range(1,101):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Alternative implementation:

```
n=1
while(n<=100):
    out=""
    if(n%3==0):
        out=out+"Fizz"
    if(n%5==0):
        out=out+"Buzz"
    if(out==""):
        out=out+str(n)
    print(out)
    n=n+1
```

Alternative implementation:

```
for i in range(100, 1):
    if i % 5 == 0 and not i % 3 == 0:
        print(i, "Buzz");
    if i % 3 == 0 and not i % 5 == 0:
        print(i, "Fizz");
    if i % 3 == 0 and i % 5 == 0:
        print(i, "FizzBuzz");
```

Alternative implementation:

```
for i in range(1, 100+1):
    out = ""
    if i % 3 == 0:
        out += "Fizz"
    if i % 5 == 0:
        out += "Buzz"
    print(out or i)
```

275 Binary digits to byte array

From the string **s** consisting of **8n** binary digit characters ('0' or '1'), build the equivalent array **a** of **n** bytes. Each chunk of 8 binary digits (2 possible values per digit) is decoded into one byte (256 possible values).

```
const size_t n = s.length() / 8;
vector<uint8_t> a(n);

for(size_t block = 0; block < n; block++)
{
    uint8_t acc = 0;
    const size_t start = block * 8;
    for(size_t offset = start; offset < start + 8; offset++)
    {
        acc = (acc << 1) + (s[offset] - '0');
    }

    a[block] = acc;
}
```

```
n = (len(s) - 1) // 8 + 1
a = bytearray(n)
for i in range(n):
    b = int(s[i * 8:(i + 1) * 8], 2)
    a[i] = b
```

277 Remove an element from a set

Remove the element **e** from the set **x**.

Explains what happens if **e** was already absent from **x**.

```
x.erase(e);
```

```
x.remove(e)
```

278 Read one line from the standard input

Read one line into the string **line**.

Explain what happens if **EOF** is reached.

```
int main() {
    for (std::string line; std::getline
(std::cin, line);) {
        std::cout << line << std::endl;
    }
    return 0;
}
```

```
line = sys.stdin.readline()
```

280 Filter map

Remove all the elements from the map **m** that **don't** satisfy the predicate **p**. Keep all the elements that **do** satisfy **p**.

Explain if the filtering happens in-place, i.e. if **m** is reused or if a new map is created.

```
for (auto it = m.begin(); it != m.end
());) {
    if (!p(it->second)) {
        it = m.erase(it);
    } else {
        ++it;
    }
}
```

```
m = {k:v for k, v in m.items() if p(v)}
```

Alternative implementation:

```
for k in list(m):
    if p(m[k]): m.pop(k)
```

284 Create a zeroed list of integers

Create a new list **a** (or array, or slice) of size **n**, where all elements are integers initialized with the value **0**.

```
std::vector<int> a(n, 0);
```

```
a = [0] * n
```

288 Check if set contains a value

Set the boolean **b** to **true** if the set **x** contains the element **e**, **false** otherwise.

```
b = x.find(e) != x.end();
```

```
b = e in x
```

289 Concatenate two strings

Create the string **s** by concatenating the strings **a** and **b**.

```
std::string a{"Hello"};
std::string b{"world"};
auto s = a + b;
```

```
s = a + b
```

299 Comment out a single line

Write a line of comments.

This line will not be compiled or executed.

```
// This is a comment.
```

```
# This is a comment
```

302 String interpolation

Given the integer **x = 8**, assign to the string **s** the value "Our sun has 8 planets", where the number **8** was evaluated from **x**.

```
s = std::format("Our sun has {} planet
s", x);
```

```
s = f'Our sun has {x} planets'
```


313 Map equality

Set ***b*** to ***true*** if the maps ***m*** and ***n*** have the same key/value entries, ***false*** otherwise.

```
b = m == n;
```

```
b = m == n
```

314 Fill array with value

Set all the elements in the array ***x*** to the same value ***v***

```
x.fill(v);
```

```
x[:] = [v] * len(x)
```

320 Test if string is empty

Set ***b*** to ***true*** if the string ***s*** is empty, ***false*** otherwise

```
b = s.empty();
```

```
b = s == ''
```

322 replace value of

variable with new one
and return old value

```
std::exchange(x, v);
```

```
old, x = x, new
```